FastAPI API Response Delay SOP

Objective

Reduce high API response time by identifying slow application code, database queries, cache misses, or downstream latency.

Scope

Applicable to FastAPI-based services, backend APIs, API gateways, DevOps teams, QA teams, and platform teams supporting production, staging, or synthetic incident workflows.

Pre-requisites

1. Access to FastAPI application logs, deployment logs, and API gateway or ingress logs.
2. Access to monitoring dashboards such as Grafana, Prometheus, APM, or cloud monitoring.
3. Permission to view application configuration, environment variables, and deployment status.
4. API testing tool such as curl, Postman, Swagger UI, or automated smoke test scripts.
5. Escalation contact for backend, DevOps, database, and security teams.

Incident Metadata

Field	Value
Ticket ID	T010
Issue	API response delay
Category	Performance
Service	FastAPI
Severity	Low
Status	Resolved
Description	API response time is high
Expected Resolution	Optimize database queries and enable caching

Related Log Evidence

No direct matching log entry was present in the provided logs CSV. Use the ticket description, application logs, API gateway logs, and monitoring dashboards to validate this incident in runtime.

Procedure

1. Confirm impacted endpoint, percentile latency, time window, and request volume.
2. Compare application latency with gateway, ingress, and client-side latency.
3. Review logs and traces for slow database queries or downstream API calls.
4. Check CPU, memory, worker count, thread pool, and event loop blocking.
5. Validate whether caching is enabled and whether cache hit rate dropped.
6. Optimize query, add indexes, enable caching, or move heavy work to background tasks.
7. Load test the corrected endpoint with representative traffic.
8. Monitor p95 and p99 latency until response time returns to baseline.

Common Commands

```
curl -i -X GET http://<host>/health
curl -i -X POST http://<host>/<endpoint> -H "Content-Type: application/json" -d @payload.json
```

```
kubectl get pods -n <namespace> | grep fastapi
kubectl logs <fastapi-pod> -n <namespace> --tail=100
kubectl describe pod <fastapi-pod> -n <namespace>
```

Troubleshooting Matrix

Issue	Possible Cause	Resolution
High response time	Slow database query	Optimize query and add indexes
Latency spike under load	Insufficient workers	Tune workers and resource limits
Only one endpoint slow	Inefficient endpoint logic	Profile and optimize code path
Delay during external call	Downstream service latency	Add timeout, retries, and fallback

Best Practices

1. Track p50, p95, and p99 latency by endpoint.
2. Avoid blocking calls inside async endpoints.
3. Cache stable reference data.
4. Use tracing to separate app, DB, and network latency.

References

FastAPI Docs | Internal Logs CSV | Tickets CSV | Monitoring Dashboards | API Gateway Logs | Backend Runbook